TP n° 1 : Quelques fonctions SQL et requêtes

Préparation de l'environnement de travail

1. Assurez-vous que votre fichier .bashrc contient bien l'environnement nécessaire pour l'exécution d'Oracle. Vous pouvez l'ouvrir (à manipuler avec précaution) avec votre éditeur préféré, par exemple vim .bashrc et enlever le commentaire # de la ligne : source /export/home/users/COMMUN/.oraclrc

2. Lancer le client d'oracle sqlplus. vous pouvez aussi utiliser le client graphique Sqldevloper, mais je vous conseille d'utiliser, au moins dans un premier temps, sqlplus.

3. Identifiez-vous avec votre login et mot de passe (fournis lors du TP) : Une fois sqlplus lancé, introduire votre login et puis votre mot de passe, ou en introduisant directement cette ligne sqlplus monLogin/monMotDePasse

4. La commande spool fichier.log permet de sauvegarder vos requêtes et leurs résultats dans le fichier fichier.log.

5. Ci-dessous quelques commandes permettant de changer le format d'affichage :

```
set colsep '|'
set linesize 200
set pagesize 20
column column_name format a30
```

Vous déposez le travail réalisé sur l'ENT.

Quelques fonctions SQL/Oracle

La table DUAL est une table d'une seule ligne et d'une seule colonne présente par défaut dans les installations du SGBD Oracle. La table contient une seule colonne de type VARCHAR2(1) appelée DUMMY qui a pour unique valeur 'X'. Elle permet d'envoyer des ordres SQL de type SELECT sans utiliser de table particulière pour tester des fonctions ou récupérer des informations indépendamment des données mais liées à la base ou à Oracle (récupérer la date système, par exemple). L'objectif dans ce qui suit est de tester certaines fonctions. La liste n'est pas exhaustive. Vous trouverez à l'adresse suivante plus de fonctions
https://docs.oracle.com/database/121/SQLRF/functions.htm#SQLRF006

Tester chacune des fonctions suivantes sur un exemple et expliquer brièvement son fonctionnement.

- Exemple de réponse :
  SYSDATE permet d'afficher la date système.
  Exemple d'utilisation :

```
SELECT SYSDATE FROM DUAL
```

- TO_CHAR ; RPAD ; LPAD ; SUBSTR ; LENGTH ; ROUND ; TRUNC ; TO_DATE ; EXTRACT

Exercice n° 1

Soit l'exemple de la gestion des cours dont le schéma relationnel correspondant est donné ci-après. Le script pour créer le schéma de la base est donnée à l'adresse suivante
https://moodlelms.univ-paris13.fr/mod/resource/view.php?id=173183 et pour la peupler, vous pouvez utiliser le script qui se trouve à l'adresse suivante
https://moodlelms.univ-paris13.fr/mod/resource/view.php?id=173184
Remarque Á ce stade, vous ne pouvez pas utiliser ces scripts, il faut lire la suite et les compléter.

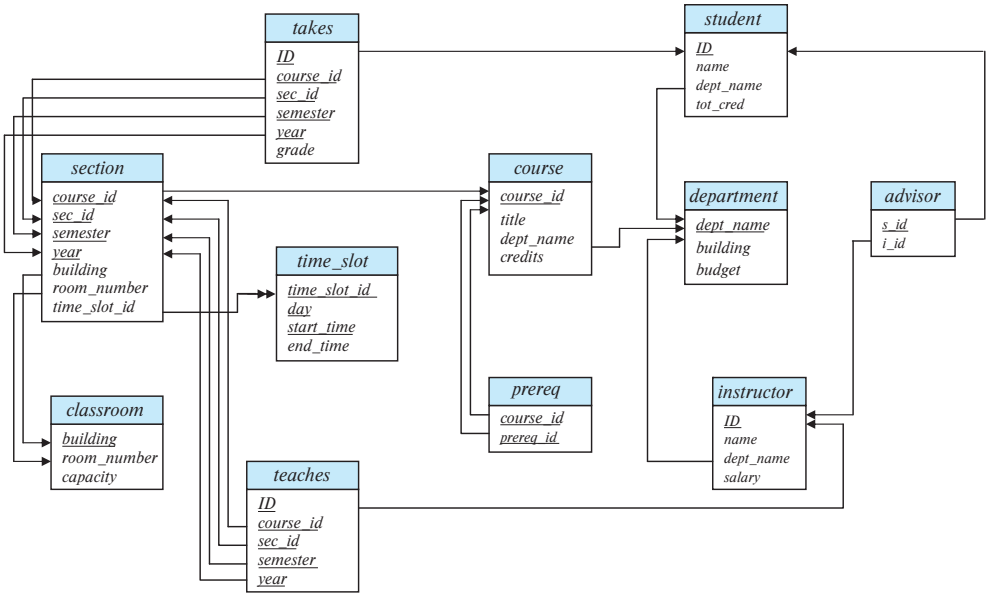

Figure 1: Schéma relationnel de la base de données de gestion des cours

1. Définir une nouvelle table **section** dont le schéma est section(**course_id, sec_id, semester, year**, *building, room_number, time_slot_id*). Tous les attributs sont de type chaînes de caractères sauf year qui est de type numérique.
   Ajouter la contrainte sur l'attribut semestre dont les valeurs appartienent à l'ensemble {'Fall', 'Winter', 'Spring', 'Summer'}. Bien placer cette définition dans le précédent script.

```
create table section(
    course_id   varchar(8),
    sec_id      varchar(8),
    semester    varchar(6)
  check (semester in ('Fall', 'Winter', 'Spring', 'Summer')),
  year      numeric(4,0),
  building    varchar(15),
  room_number   varchar(7),
  time_slot_id    varchar(4),
  primary key (course_id, sec_id, semester, year),
  foreign key (course_id) references course,
  foreign key (building, room_number) references classroom
  );
```

2. Déterminer le diagramme entité-association de cette base de données. Rappel : avant d'interroger une base de donnée, prendre le temps de bien comprendre sa structure et avoir à l'esprit qu'elle ne modélise pas la réalité.

3. Procéder à la création de la base de données et la peupler. Je vous rappelle que le lancement du script *test.sql* sous *sqlplus* se fait comme suit

```
SQL>@test
```

4. Insérer un nouveau cours dont l'identifiant est BIO-101, intitulé *Intro. to Biology*, assuré par le département *Biology* et son crédit est de 4.

```
insert into course values ('BIO-101', 'Intro. to Biology', 'Biology', '4');
```

Exercice n° 2

1. Afficher la structure de la relation section et son contenu (cours proposés).

```sql
DESC section;
SELECT *
FROM section;
```

2. Afficher tous les renseignements sur les cours que l'on peut programmer (relation course).

```sql
SELECT *
FROM course;
```

3. Afficher les titres des cours et les départements qui les proposent.

```sql
SELECT title, dept_name
FROM course;
```

4. Afficher les noms des départements ainsi que leur budget.

```sql
SELECT dept_name, budget
FROM department;
```

5. Afficher tous les noms des enseignants et leur département.

```sql
SELECT name, dept_name
FROM teacher;
```

6. Afficher tous les noms des enseignants ayant un salaire supérieur strictement à 65.000 $.

```sql
SELECT name
FROM teacher
WHERE salary > 65000;
```

7. Afficher les noms des enseignants ayant un salaire compris entre 55.000 $ et 85.000 $.

```sql
SELECT name
FROM teacher
WHERE salary between  55000 and 85000;
```

8. Afficher les noms des départements, en utilisant la relation teacher et éliminer les doublons.

```sql
SELECT DISTINCT dept_name
FROM teacher;
```

9. Afficher tous les noms des enseignants du département informatique ayant un salaire supérieur strictement à 65.000 $.

```sql
SELECT name
FROM teacher
WHERE salary > 65000 and dep_name = 'Comp. Sci.';
```

10. Afficher tous les renseignements sur les cours proposés au printemps 2010 (relation section).

```sql
SELECT *
FROM section
WHERE semester='Spring';
```

11. Afficher tous les titres des cours dispensés par le département informatique qui ont plus de trois crédits.

```sql
SELECT title
FROM course
WHERE dept_name='Comp. Sci.' AND credit>3;
```

---

12. Afficher tous les noms des enseignants ainsi que le nom de leur département et les noms des bâtiments qui les hébergent.

```sql
SELECT teacher.name, teacher.dept_name, department.building
FROM teacher, department
WHERE teacher.dept_name = department.dept_name;
```

13. Afficher tous les étudiants ayant suivi au moins un cours en informatique.

```sql
SELECT distinct student.name
FROM student, takes, course
WHERE student.id = takes.id and takes.course_id = course.
course_id and course.dept_name = Comp. Sci. ;
```

14. Afficher les noms des étudiants ayant suivi un cours dispensé par un enseignant nommé Eistein (éliminer les doublons).

```sql
SELECT distinct student.name
FROM student, teacher, takes, teaches
WHERE student.id = takes.id and takes.course_id = teaches.
course_id and takes.sec_id = teaches.sec_id and takes. semester = teaches.semester
     and takes.year = teaches.year and
teaches.id = teacher.id and teacher.name = Einstein ;
```

15. Afficher tous les identifiants des cours et les enseignants qui les ont assurés.

```sql
SELECT name, course_id
FROM teacher, teaches
WHERE teacher.id = teaches.id;
```

---

16. Afficher le nombre d'inscrits pour chaque enseignement proposé au printemps 2010.

```sql
SELECT takes.course_id, takes.sec_id, takes.semester, takes. year, count(*)
FROM takes
WHERE takes.semester = Spring and takes.year = 2010
GROUP BY takes.course_id, takes.sec_id, takes.semester, takes.
year;
```

17. Afficher les noms des départements et les salaires maximum de leurs enseignants.

```sql
 SELECT dept_name, max(salary) FROM teacher
GROUP BY dept_name;
```

18. Afficher le nombre d'inscrits pour chaque enseignement proposé.

```sql
SELECT takes.course_id, takes.sec_id, takes.semester, takes. year, count(*)
FROM takes
GROUP BY takes.course_id, takes.sec_id, takes.semester, takes.
year;
```

19. Afficher le nombre total de cours qui ont eu lieu dans chaque bâtiment, pendant l'automne 2009 et le printemps 2010.

```sql
SELECT building, count(*)
FROM section
WHERE (semester, year) IN ( ( Fall , 2009), ( Spring , 2010) ) GROUP BY building;
```

20. Afficher le nombre total de cours dispensés par chaque département et qui ont eu dans le même bâtiment qui l'abrite.

```sql
SELECT department.dept_name, count(*)
FROM section, department, teacher, teaches
WHERE (section.course_id, section.sec_id, section.semester,
section.year) = (teaches.course_id, teaches.sec_id, teaches. semester, teaches.
    year) and
teaches.id = teacher.id and teacher.dept_name = department.dept_name and
    department.building = section. building
GROUP BY department.dept_name;
```

21. Afficher les titres des cours proposés et qui ont eu lieu et les enseignants qui les ont assurés.

```sql
SELECT course.title, teacher.name
FROM section, teacher, teaches, course
WHERE (section.course_id, section.sec_id, section.semester,
section.year) = (teaches.course_id, teaches.sec_id, teaches. semester, teaches.
    year) and teaches.id = teacher.id and section.course_id = course.course_id
ORDER BY course.title;
```

22. Afficher le nombre total de cours qui ont eu lieu pour chacune des période *Summer, Fall et Spring*.

```sql
SELECT section.semester, count(*) FROM section
GROUP BY section.semester
```

23. Afficher pour chaque étudiant le nombre total de crédits qu'il a obtenu, en suivant des cours qui n'ont pas été proposé par son département.

```sql
SELECT student.name, sum(credits)
FROM student, course, takes
WHERE student.id = takes.id and takes.course_id = course.
course_id and student.dept_name != course.dept_name GROUP BY student.name;
```

24. Pour chaque département, afficher le nombre total de crédits des cours qui ont eu lieu dans ce département.

```sql
SELECT section.building, sum(course.credits) FROM section, course
WHERE section.course_id = course.course_id GROUP BY section.building;
```